University of Brescia, Italy
Machine Learning & Data Mining

Concept Learning

Prof. Alfonso Gerevini

UniBS

Concept Learning

A **concept** can be seen as a subset of a larger set of objects/events/ entities (birds $\subset$ animals)

Concept learning $\Rightarrow$ *Inferring a boolean-valued function from training examples of its input and output.*

The learned function is derived from a set of possible functions defining the *hypothesis space*

In this lecture:

> A simple approach and basic algorithms for concept learning assuming no noise in training data (illustrates key concepts)

Training Examples for EnjoySport on a Day

Sky	Temp	Humid	Wind	Water	Forecst	EnjoySpt
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Examples are positive or negative

What is the general concept?

$EnjoySport : Sky \times Temp \times Humid \times Wind \times Water \times Forecst \rightarrow \{Yes, No\}$

$Sky = \{S, C, R\}, Temp = \{W, C\}, Humid = \{N, H\}, Wind = \{S, W\}$
$Water = \{W, C\}, Forecst = \{S, C\}$

Notation

c: target function $c : X \to \{0, 1\}$

X: instance space

$x \in X$: one instance (= one day described by its feature values)

$D = \{\langle x_i, c(x_i)\rangle\}$: training set

$c(x)$: value of the target function over x (*true value*)

H: hypothesis space

$h \in H$: one hypothesis (an approximation of c)

$h(x)$: estimation of h over x (*predicted or estimated value*)

Learning task

Given a training set $D = \{\langle x_i, c(x_i)\rangle\}$, find the *best* approximation $h^* \in H$ of the target function (or *target concept*) $c : X \rightarrow \{0, 1\}$.

$c(x) = 1/0 \Rightarrow \text{EnjoySpt}(x) = \text{YES/NO}$

Best approximation: $h^*(x_i) = c(x_i)$ for all $i = 1 \ldots |D|$

Steps:

1. Define the hypothesis space H (i.e., a representation of the hypotheses)
2. Define a performance metric to determine the *best* approximation
3. Define an appropriate algorithm

Representing hypotheses

Many possible representations.
Let's consider h as <u>conjunction of constraints on attributes</u>.

Each constraint can be

- a specfic value (e.g., $Water = Warm$)
- don't care (e.g., "$Water =?$")
- no value allowed (e.g.,"Water=$\emptyset$")
- *disjunction of attribute values (not used in our example!)*

For example,

	Sky	Temp	Humid	Wind	Water	Forecst
h =	$\langle Sunny,$	?,	?,	$Strong,$	?,	$Same\rangle$

Evaluating instances on hypotheses

An instance x is **classified** as a **positive example** by h ($h(x) = 1$) if and only if x satisfies all the constraints in h.

Examples:

$$\left.\begin{array}{l} h = \langle Sunny, ?, ?, Strong, ?, Same\rangle \\ x = \langle Sunny, Warm, Normal, Strong, Warm, Same\rangle \end{array}\right. \rightarrow h(x) = 1$$

$$\left.\begin{array}{l} h = \langle Sunny, ?, ?, Strong, ?, Same\rangle \\ x = \langle Rainy, Cold, High, Strong, Warm, Change\rangle \end{array}\right. \rightarrow h(x) = 0$$

$$\left.\begin{array}{l} h = \langle Sunny, Cold, High, Strong, Warm, Change\rangle \\ x = \langle Sunny, Cold, High, Strong, Warm, Change\rangle \end{array}\right. \rightarrow h(x) = 1$$

Performance measure

Given a training set $D = \{\langle x_i, c(x_i)\rangle\}$ and a hypothesis $h \in H$, a performance measure is based on evaluating $c(x_i) = h(x_i)$ for all $x_i \in D$.

Why?

$\Rightarrow$ **Inductive learning hypothesis:** *Any hypothesis found to approximate the target function well over a sufficiently large set of training examples will also approximate the target function well over other unobserved examples.*

Prototypical Concept Learning Task

Given:

- Instances X: Possible days, each described by the attributes/features *Sky, Temperature, Humidity, Wind, Water, Forecast*
- Target function $c = EnjoySport : X \to \{0, 1\}$
- Hypotheses H: Conjunctions of constraints on the attributes (e.g., $\langle ?, Cold, High, ?, ?, ? \rangle$)
- Training examples $D = \{\langle x_1, c(x_1)\rangle, \ldots, \langle x_m, c(x_m)\rangle\}$, positive and negative examples of the target function

Determine:

- A hypothesis h in H such that $h(x) = c(x)$, $\forall x \in D$.

Concept Learning as search

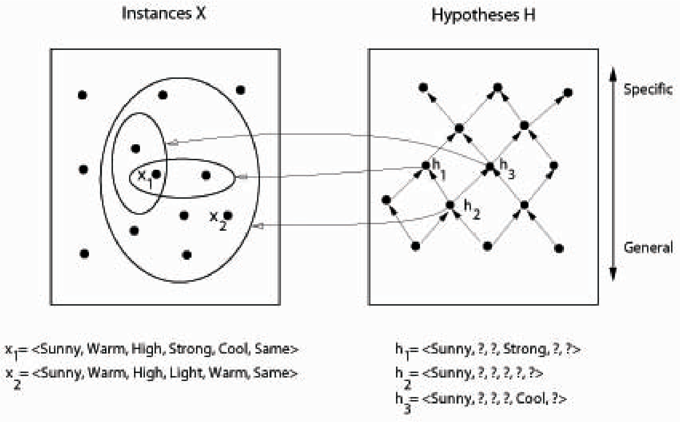
- Searching through a (exponentially) large space of hypotheses.
- Find the hypothesis that best fits the training examples.
- In the *EnjoySport* learning task, the instance set X has 96 distinct instances and the hypotheses set H has 973 semantically distinct hypotheses.

  Why these numbers?

- Note: Most practical learning tasks have much larger hypothesis spaces!

Instance, Hypotheses, and More-General-Than

General-to-specific (partial) ordering of hypotheses



More-General-Than Definition

Let h_j and h_k be boolean valued functions defined over X.

h_j is **more_general_than_or_equal_to** h_k ($h_j \geq_g h_k$) if and only if

$$(\forall x \in X)[(h_k(x) = 1) \rightarrow (h_j(x) = 1)]$$

h_j is **more_general_than** h_k ($h_j >_g h_k$) if and only if

$$(h_j \geq_g h_k) \wedge (h_k \not\geq_g h_j)$$

$\Rightarrow$ *The relation provides a useful structure over the hypothesis space that can be exploited by the learning algorithms*

FIND-S **Algorithm**

Uses **more_general_than(_or_equal)** to organise the search of a good h

1. Initialize h to the most specific hypothesis in H

2. For each positive training instance $x \in D$
   - For each attribute constraint a_i in h

     If the constraint a_i in h is satisfied by x

     Then do nothing

     Else replace a_i in h by the next more general constraint that is satisfied by x

3. Output hypothesis h

Execution of Find-S

0. $h \leftarrow < \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset >$

1. Obs. $< S, W, N, S, W, S, Yes > \Rightarrow \; h \leftarrow < S, W, N, S, W, S >$

2. Obs. $< S, W, \mathbf{H}, S, W, S, Yes > \Rightarrow \; h \leftarrow < S, W, ?, S, W, S >$

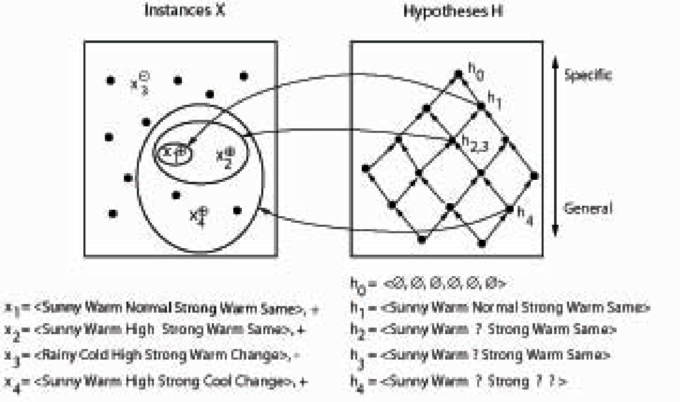
3. Obs. $< R, C, H, S, W, C, \mathbf{No} > \Rightarrow$ do nothing.

4. Obs. $< S, W, H, S, \mathbf{C}, \mathbf{C}, Yes > \Rightarrow \; h \leftarrow < S, W, ?, S, ?, ? >$

Assume Water $\in \{W, C, M\}$ and a new Obs.:

5. Obs. $< S, W, H, S, \mathbf{M}, C, \mathbf{NO} > \Rightarrow$ What happens? Why?

Hypothesis Space Search by FIND-S



Complaints about Find-S

- Can't tell whether it has learned the correct target concept
(Is the output h the only one consistent with training data?)
- Can't tell when training data inconsistent
(Noisy or inconsistent examples)
- Picks a maximally specific h
(If there are multiple consistent h, why choosing a max specific one?)
- Depending on H, there might be several! (Then which one?)

Version Spaces

A hypothesis h is **consistent** *with a set of training examples D of target concept c if and only if $h(x) = c(x)$ for each training example $\langle x, c(x) \rangle$ in D.*

$$Consistent(h, D) \equiv (\forall x \in D)\ h(x) = c(x)$$

The **version space**, *$VS_{H,D}$, with respect to hypothesis space H and training examples D, is the subset of hypotheses from H consistent with all training examples in D.*

$$VS_{H,D} \equiv \{h \in H | Consistent(h, D)\}$$

List-Then-Eliminate **Algorithm**

Key Idea: Output a description of the set of *all* hypotheses consistent with the training examples

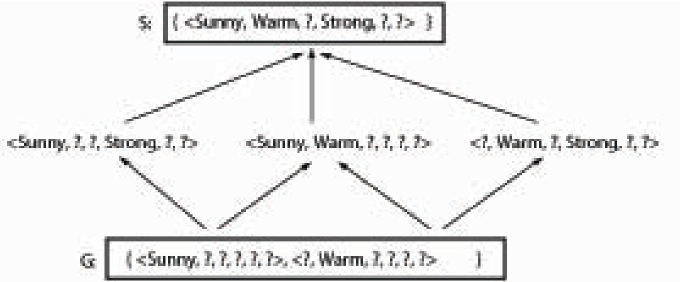
1. *VersionSpace* $\leftarrow$ a list containing every hypothesis in H
2. For each training example, $\langle x, c(x) \rangle$
   remove from *VersionSpace* any hypothesis h for which $h(x) \neq c(x)$
3. Output the list of hypotheses in *VersionSpace*

Note: enumerating all the hypotheses! (+ the noise data problem remains)

A compact representation of Version Spaces

- Using general-to-specific ordering.
- Version Space is represented by two boundary sets of hypotheses (generic and specific) that delimit the version space within the partially ordered hypothesis space.

Example of Version Space



Note:

- arrows indicate **more_general_than** relations;
- S above is the result of FIND-S algorithm in the previous example;
- All six hypotheses consistent with training data

Representing Version Spaces

Only the sets of most general and specific hypotheses are represented

The **General boundary**, G, of version space $VS_{H,D}$ is the set of its maximally general members

$$G \equiv \{g \in H | Consistent(g, D) \wedge (\neg \exists g' \in H)[(g' >_g g) \wedge Consistent(g', D)]\}$$

The **Specific boundary**, S, of version space $VS_{H,D}$ is the set of its maximally specific members

$$S \equiv \{s \in H | Consistent(s, D) \wedge (\neg \exists s' \in H)[(s >_g s') \wedge Consistent(s', D)]\}$$

Version Space Representation Theorem

Every member of the version space lies between boundaries G and S:

$$VS_{H,D} = \{h \in H | (\exists s \in S)(\exists g \in G)(g \geq_g h \geq_g s)\}$$

where $x \geq_g y$ means x is more general or equal to y

$\Rightarrow$ *The version space is precisely* $G \cup S \cup \{$ *hypotheses between* G *and* S *according to* $\geq_g$ $\}$

Proof: see Mitchell's book p. 32

Candidate Elimination Algorithm

$G \leftarrow$ maximally general hypotheses in H (which one?)
$S \leftarrow$ maximally specific hypotheses in H (which one?)
For each training example d, do

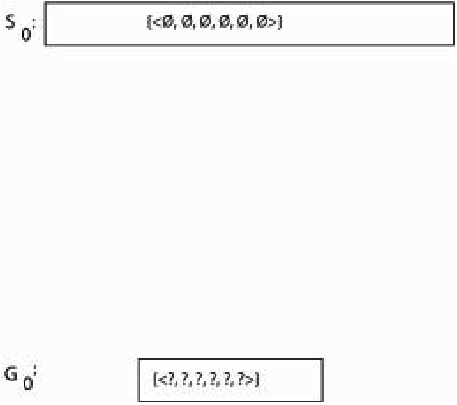
- If d is a positive example *[generalisation of S]*
  - *Remove from G any hypothesis inconsistent with d*
  - *For each hypothesis s in S that is not consistent with d*
    - *Remove s from S*
    - *Add to S all minimal generalizations h of s such that*
      - h is consistent with d, and
      - some member of G is more general than h
    - *Remove from S any hypothesis that is more general than another hypothesis in S*

Candidate Elimination Algorithm

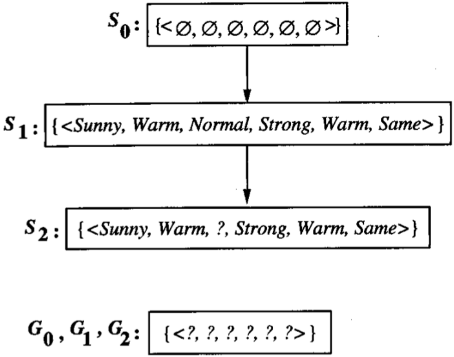
Complementary part for G

- If d is a negative example *[specialisation of G]*
  - *Remove from S any hypothesis inconsistent with d*
  - *For each hypothesis g in G that is not consistent with d*
    - *Remove g from G*
    - *Add to G all minimal specializations h of g such that*
      - h is consistent with d, and
      - some member of S is more specific than h
    - *Remove from G any hypothesis that is less general than another hypothesis in G*

Example Trace (1)



Example Trace (2)



Training examples:

1. *<Sunny, Warm, Normal, Strong, Warm, Same>, Enjoy Sport = Yes*
2. *<Sunny, Warm, High, Strong, Warm, Same>, Enjoy Sport = Yes*

Example Trace (3)

S_2, S_3: { <*Sunny, Warm, ?, Strong, Warm, Same*> }

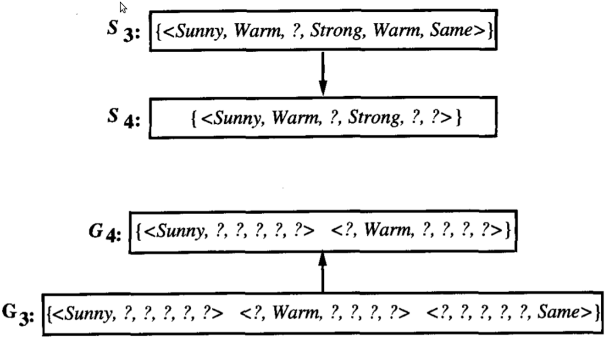
G_3: {<*Sunny, ?, ?, ?, ?, ?*> <*?, Warm, ?, ?, ?, ?*> <*?, ?, ?, ?, ?, Same*>}

G_2: {<*?, ?, ?, ?, ?, ?*>}

Training Example:

3. <*Rainy, Cold, High, Strong, Warm, Change*>, *EnjoySport=No*

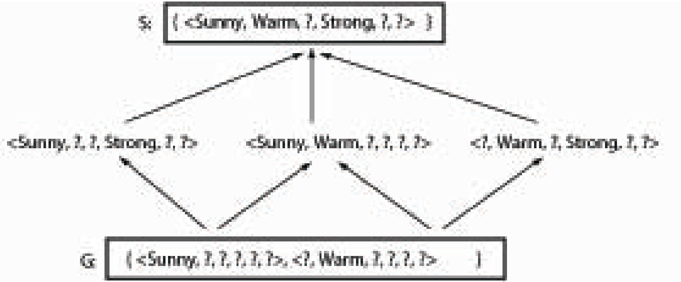
Example Trace (4)



Training Example:

4. <*Sunny, Warm, High, Strong, Cool, Change*>, *EnjoySport* = *Yes*

Final Version Space



Exercise 2.4

Instance space X: integer points in x,y plane (integers ≥ 0)
Hypotheses H: rectangles (areas) in the x,y plane parallel to the axes
Data set D: positive and negative examples of points belonging to a rectangle
Determine the version space (S and G boundaries) of data set
$D = \{\langle(1,1),+\rangle, \langle(4,2),+\rangle, \langle(2,4),-\rangle, \langle(0,0),-\rangle, \langle(5,2),-\rangle,\}$

Remarks on Candidate-Elimination Algorithm

The Candidate-Elimination Algorithm will converge toward the hypothesis that correctly describes the target concept if (necessary but not sufficient condition):

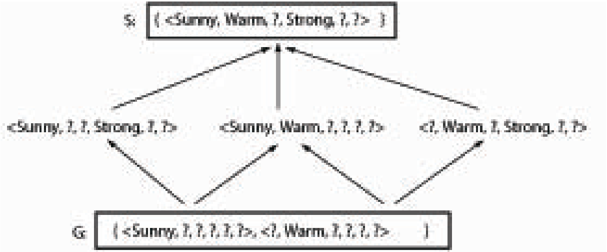
1. there are no errors in the training examples;
2. there is an hypothesis in H that correctly describes the target concept.

The target concept is exactly learned when the S and G boundary sets *converge to a single identical hypothesis*

Choosing the next instance

- **On-line learning**: the learner constructs the instance and asks an oracle or teacher to give the correct classification
- Assuming to have a teacher for on-line training, what would be a good instance to consider next (*query*)?
- The **optimal query strategy** for a concept learner is to generate instances that satisfy exactly half of the hypothesis in the current version space.
- If this is possible, we can find the target concept with $\lceil log_2 |VS| \rceil$ queries.

Classification of new instances: How?



$$x_1 = \langle Sunny\ \ Warm\ \ Normal\ \ Strong\ \ Cool\ \ Change \rangle$$

$$x_2 = \langle Rainy\ \ Cool\ \ Normal\ \ Weak\ \ Warm\ \ Same \rangle$$

$$x_3 = \langle Sunny\ \ Warm\ \ Normal\ \ Weak\ \ Warm\ \ Same \rangle$$

Classification of new instances

$$x_1 = \langle Sunny\ \ Warm\ \ Normal\ \ Strong\ \ Cool\ \ Change \rangle \rightarrow Yes$$

Since it is classified as a positive instance by all the hypotheses.
It is sufficient to verify that the instance is classified as **positive by every member of** S. Why?

$$x_2 = \langle Rainy\ \ Cool\ \ Normal\ \ Weak\ \ Warm\ \ Same \rangle \rightarrow No$$

Since it is classified as a negative instance by all the hypotheses.
It is sufficient to verify that the instance is classified as **negative by every member of** G. Why?

Classification of new instances

$$x_3 = \langle \textit{Sunny Warm Normal Weak Warm Same} \rangle \rightarrow ?$$

since there are some hypotheses that classify the instance as positive and some others that classify it as negative.

Note: Find-S would classify this new instance as negative! Why?

$\Rightarrow$ Majority vote rule (with a possible *confidence rate*)

A Biased Learner

- The space H we have considered so far contains only a small set of possible hypotheses (only conjunctions of attribute values!).
- The choice of such a representation for the hypotheses represents the **inductive bias** of the learner.
- The considered space (**inductive bias**) cannot represent *disjunctive target concepts* such as "Sky = Sunny or Sky = Cloudy".

A Biased Learner: example

Consider our inductive bias and the training examples

- x1: [S, W, N, S, C, CH, YES]
- x2: [C, W, N, S, C, CH, YES]

- x3: [R, W, N, S, C, CH, **NO**]

Maximal specific hypothesis S_2 after processing x1 and x2 $=$???
Which is the problem then with x3?

An UNBiased Learner

Idea: *Choose H that expresses every teachable concept* (H is capable to represent every subset of X, i.e., the power set of X)

Consider $H' =$ disjunctions, conjunctions, negations over the previously defined H. E.g.,

$$\langle Sunny\ Warm\ Normal\ ?\ ?\ ?\rangle \ \vee\ \neg\langle ?\ ?\ ?\ ?\ ?\ Change\rangle$$

$|H'| = 2^{96} \simeq 10^{28}$ (while $|H| = 973$)

$96 =$ size of the instance space. H' very LARGE.

An UNBiased Learner

Consider

$$D = \{< x_1, 1 >, < x_2, 1 >, < x_3, 1 >, < x_4, 0 >, < x_5, 0 >\}$$

What are S, G in this case?

S $\leftarrow$

G $\leftarrow$

x_i = vector of attribute values for instance

An UNBiased Learner

$$D = \{< x_1, 1 >, < x_2, 1 >, < x_3, 1 >, < x_4, 0 >, < x_5, 0 >\}$$

$S \leftarrow \{(x_1 \vee x_2 \vee x_3)\} \equiv$ disjunction of positive examples
$G \leftarrow \{\neg(x_4 \vee x_5)\} \equiv$ negated disjunction of negative examples

The only instances unambiguously classified are the training examples!

Each unobserved instance will be classified positive by half the hypotheses in the version space and will be classified negative by the other half.

Unbiased vs. Biased Learning

A learner that makes no a priori assumptions regarding the identity of the target concept has no rational basis for classifying any unseen instances.

Our previous learners were able to correctly classify new instances (if there are no errors in the training set and the target concept is included in H) because of the assumption that the target concept can be represented as conjunction of attributes.

If this assumption is incorrect, at least some instances will be erroneously classified.

Inductive Bias of C-E Algorithm using H

The *inductive bias* of the Candidate-Elimination Algorithm using hypothesis space H is that the target concept c is included in H.

Inductive Bias of Find-S Algorithm using H

The *inductive bias* of the FIND-S Algorithm using hypothesis space H is that the target concept c is included in H and that the solution is a maximally specific hypothesis.

Three Learners with Different Biases

1. *Rote learner:* Store examples, Classify x (YES/NO) if and only if it matches previously observed example.
2. Version space Candidate-Elimination algorithm (all members of the version space agree on the classification)
3. FIND-S (most specific consistent hypothesis)

Learners with stronger inductive bias classify a greater proportion of unseen instances.

How are these learners ranked in terms of their inductive bias?

Summary Points

1. Concept learning as search through hypothesis space
2. General-to-specific ordering of hypotheses
3. Version space candidate elimination algorithm
4. S and G boundaries characterize learner's uncertainty
5. Inductive jumps possible only if learner is biased